

AWS & TERRAFORM

Complete Interview Cheat Sheet

Quick Review • Interview Q&A • Project Ideas

PART 1 — TERRAFORM

Terraform Core Concepts

What is Terraform?

Terraform is a multi-cloud Infrastructure as Code (IaC) tool by HashiCorp. You write .tf files describing what infrastructure you want, and Terraform creates/updates/destroys it automatically.

Key advantage over others: Works with AWS, Azure, GCP, and on-prem from the same code.

Essential Terminology

Term	What It Is	Simple Analogy
Provider	Plugin to interact with a cloud platform (aws, azure, google)	Like a driver for your cloud
Resource	Actual infra component (EC2, S3, VPC, etc.) you create	The actual thing being built
Module	Reusable packaged Terraform code	Like a function in programming
Variable	Input placeholder so config is reusable	Like function parameters
Output	Value exposed after apply (e.g., public IP)	Like a function return value
State File	terraform.tfstate — tracks what Terraform created	Like a receipt of everything created
Plan	Preview of what will change — terraform plan	Dry-run before actual action
Apply	Actually create/update infra — terraform apply	The actual execution
Workspace	Separate state per environment (dev/stage/prod)	Like Git branches for infra
Remote Backend	Store state file in S3 instead of locally	Shared team storage for state
tfvars file	.tfvars — pass variable values without hardcoding in code	Config file separate from code

Terraform Workflow (4 Commands)

Command	What It Does
terraform init	Download providers, set up backend — run this first always
terraform plan	Show what will be created/changed/destroyed — always review this
terraform apply	Execute the plan — creates actual infrastructure
terraform destroy	Delete everything Terraform created — be very careful

Provider Configuration

```
provider "aws" { region = "us-east-1" }
```

Multiple providers: Use alias keyword for multi-region or multi-cloud.

```
provider "aws" { alias = "us-west-2"; region = "us-west-2" }
resource "aws_instance" "example" { provider = "aws.us-west-2"; ... }
```

Variables & tfvars

Declare variables in variables.tf, set values in terraform.tfvars or pass with -var-file.

```
variable "instance_type" { description = "EC2 type"; type = string; default = "t2.micro"
}
# terraform.tfvars instance_type = "t2.large"
terraform apply -var-file=prod.tfvars
```

Interview Tip

tfvars files keep secrets and environment-specific values OUT of your code. Never commit secrets to Git — use environment variables or Vault instead.

Modules (Day 3 Project Pattern)

Create a ./modules/ec2_instance/ folder with its own main.tf, variables.tf, outputs.tf. Reference it from root:

```
module "ec2_instance" { source = "../modules/ec2_instance" ami_value = var.ami_value
instance_type_value = var.instance_type_value }
```

Output from module is accessed as: module.ec2_instance.public-ip-address

Remote Backend — S3 (Day 4)

Problem: State file on your laptop = team can't collaborate + risk of loss. Solution: store in S3.

Chicken-and-Egg Problem

You CANNOT create an S3 bucket and use it as backend in the same apply. First create the bucket (no backend block), then add backend block and run terraform init again.

```
terraform { backend "s3" { bucket = "my-terraform-state" key =
"project/terraform.tfstate" region = "us-east-1" dynamodb_table = "terraform-
lock" # prevents simultaneous applies encrypt = true } }
```

Workspaces — Multi-Environment (Day 6)

Same code, different state per environment. Each workspace has isolated state.

```
terraform workspace new dev terraform workspace new prod terraform workspace select prod
terraform apply
```

Use lookup() to pick env-specific values:

```
variable "instance_type" { default = { dev = "t2.micro", stage = "t2.medium", prod =
"t2.xlarge" } } instance_type = lookup(var.instance_type, terraform.workspace,
"t2.micro")
```

Provisioners (Day 5)

Provisioner	What It Does
file	Copies files from your machine to EC2 over SSH
remote-exec	Runs commands on the remote EC2 over SSH (install packages, start services)
local-exec	Runs commands on YOUR local machine (useful for scripts after provisioning)

Key Point

remote-exec with nohup command & — the & runs in background so Terraform doesn't hang waiting. nohup keeps the process alive after SSH disconnects.

Conditional Expressions

```
count = var.create_instance ? 1 : 0    # create or skip resource cidr = var.env == "prod"  
? var.prod_cidr : var.dev_cidr
```

Securing Terraform

- Mark sensitive variables: sensitive = true (hides from console output)
- Use environment variables: export TF_VAR_db_password=secret
- HashiCorp Vault integration for secrets (Day 7 project)
- Remote backend with encryption + S3 bucket policy

PART 2 — AWS CORE SERVICES

1. IAM — Identity & Access Management

The gatekeeper for your entire AWS account. Controls WHO can do WHAT with WHICH resources.

Term	What It Is
User	Individual identity with long-term credentials (username + password or access keys)
Group	Collection of users sharing same permissions — attach policies to groups not users
Role	Temporary identity assumed by EC2, Lambda, or cross-account. No password — uses STS tokens
Policy	JSON document defining allowed/denied actions. Attached to users/groups/roles
MFA	Multi-Factor Auth — mandatory on root account and admin users

Policy JSON Structure

```
{  "Effect": "Allow",           // or "Deny"  "Action": ["s3:GetObject"], // which API
  "Resource": "arn:aws:s3:::mybucket/*" // which resource }
```

Interview Trap

Role vs User: A User has permanent credentials. A Role is temporary — it's ASSUMED by a service (EC2, Lambda) and issues short-lived credentials via STS. Always use Roles for service-to-service access, never hardcode access keys.

- Best practice: Least privilege — give only what is needed
- Never use root for daily tasks
- Rotate access keys regularly

2. EC2 — Elastic Compute Cloud

Resizable virtual servers in the cloud. You choose OS, CPU, RAM, storage, networking.

Instance Family	Use Case	Examples
General Purpose	Web servers, dev/test	t3.micro, m5.large
Compute Optimized	Batch processing, gaming	c5.large, c6g.xlarge
Memory Optimized	Redis, databases, analytics	r5.xlarge, x1e.32xlarge
Storage Optimized	Data warehousing, Hadoop	i3.large, d2.8xlarge
Accelerated (GPU)	ML training, video rendering	p3.2xlarge, g4dn.xlarge

Pricing Models

Model	Description	Best For
On-Demand	Pay per hour/second, no commitment	Variable workloads, testing
Reserved	1 or 3 year commitment — up to 75% cheaper	Stable predictable workloads
Spot	Bid on unused capacity — up to 90% cheaper, can be interrupted	Fault-tolerant batch jobs

Dedicated	Physical server dedicated to you	Compliance, licensing requirements
-----------	----------------------------------	------------------------------------

Key Concepts

- AMI: Template/snapshot used to launch instances
- EBS: Block storage attached to EC2 (like a hard drive)
- Elastic IP: Static public IP that survives instance restarts
- User Data: Scripts run automatically on first launch
- SSH access: `chmod 600 key.pem` then `ssh -i key.pem ubuntu@<ip>`
- App not reachable? Check Security Group inbound rules — add port 80/443

3. VPC — Virtual Private Cloud

Your own isolated network inside AWS. Think of it as your private data center in the cloud.

Request Flow: Internet → Your App

Step	Component	Role
1	Internet Gateway (IGW)	Entry/exit door of the VPC — connects to internet
2	Public Subnet	Subnet with route to IGW — resources here have public IPs
3	Load Balancer (ALB)	Receives internet traffic, distributes to private instances
4	Route Table	Traffic rules: 0.0.0.0/0 → IGW for public, local for private
5	NACL	Subnet-level stateless firewall — evaluated by rule number
6	Private Subnet	No internet route — EC2 instances, databases live here
7	Security Group	Instance-level stateful firewall — controls ports/protocols
8	NAT Gateway	Lets private instances make OUTBOUND internet calls (e.g., apt install)

Security Groups vs NACLs

Feature	Security Group	NACL
Level	Instance level	Subnet level
State	Stateful — response traffic auto-allowed	Stateless — must allow both directions
Rules	Allow only	Allow AND Deny
Rule Order	All rules evaluated together	Lowest number = highest priority
Use Case	Per-instance fine-grained control	Block traffic at subnet boundary

Interview Trap

NACLs are stateless. If you allow inbound port 80, you ALSO need to allow outbound ephemeral ports (1024-65535) for the response. Security Groups don't have this problem — they're stateful.

- Public subnet = has route to IGW in route table
- Private subnet = no IGW route, uses NAT Gateway for outbound
- VPC CIDR /16 = 65,536 IPs. Subnet /24 = 256 IPs (251 usable)

4. S3 — Simple Storage Service

Object storage — infinitely scalable, 99.999999999% (11 nines) durability.

Storage Class	Use Case	Retrieval
Standard	Frequently accessed	Milliseconds
Standard-IA	Infrequent but needs fast retrieval	Milliseconds
Glacier Instant	Archive with instant retrieval	Milliseconds
Glacier Flexible	Archive, occasional access OK	Minutes to hours
Glacier Deep	Long-term, rarely accessed	Hours
Intelligent-Tiering	Auto-moves to right tier	Varies

Key Features

- Versioning: Keep all versions of files — protects from accidental deletes
- Bucket policies: JSON-based access control
- Static website hosting: Serve HTML/CSS/JS directly from S3
- Terraform remote backend: Store tfstate here for team sharing

5. CloudWatch — Monitoring & Observability

Pillar	What It Does
Metrics	Numerical data over time — CPU, requests, errors
Logs	Text output from apps/services — centralized in Log Groups
Alarms	Trigger action when metric breaches threshold (SNS, Auto Scaling)
Events/EventBridge	React to AWS environment changes (EC2 state change, schedule)

Custom metrics: Push memory/disk usage via CloudWatch agent (not available by default)

Scheduled Lambda: EventBridge cron expression `cron(0 2 * * ? *)` = run daily at 2 AM

6. Lambda — Serverless Compute

Upload code, define a trigger, AWS handles everything. No servers to manage.

- Event-driven: S3 upload, API Gateway, CloudWatch Events, SQS
- Pay per invocation only — zero idle cost
- Auto-scales to thousands of concurrent executions
- Max timeout: 15 minutes per invocation
- Needs IAM execution role to access other AWS services

Real Project (from your notes)

EBS Snapshot Cleanup Lambda: List all snapshots → List running EC2s → Delete snapshots with no volume or volume not attached to any running instance. Schedule with EventBridge cron to run nightly.

7. ECR — Elastic Container Registry

Private Docker registry inside AWS. Like Docker Hub but with native IAM auth.

```
aws ecr get-login-password --region us-east-1 | docker login --username AWS --password-stdin <acct>.dkr.ecr.us-east-1.amazonaws.com docker build -t my-app . docker tag my-app:latest <acct>.dkr.ecr.us-east-1.amazonaws.com/my-repo:latest docker push <acct>.dkr.ecr.us-east-1.amazonaws.com/my-repo:latest
```

- Enable immutable tags: Prevent tag from being overwritten
- Enable push scan: Auto-scan for CVEs on every push
- Lifecycle policies: Auto-delete old/untagged images to cut cost

8. ECS — Elastic Container Service

Component	Description	K8s Equivalent
Cluster	Logical grouping of infrastructure	Cluster
Task Definition	Blueprint: image, CPU, memory, ports, env vars	Pod spec
Task	A running instance of a Task Definition	Pod
Service	Maintains N running tasks + load balancer integration	Deployment + Service

Fargate vs EC2 launch type: Fargate = no infra management, auto-scales, costs more per unit. EC2 = you manage instances, cheaper at scale.

9. EKS — Elastic Kubernetes Service

Managed Kubernetes — AWS handles the control plane (API server, etcd, scheduler). You manage worker nodes.

IRSA — IAM Roles for Service Accounts

Pods need AWS access without storing access keys inside containers. IRSA solves this:

- Configure OIDC provider for EKS cluster
- Create IAM Role with trust policy pointing to K8s service account
- Annotate service account with IAM role ARN
- Pods using that account automatically get temporary AWS credentials

Interview Key

IRSA is far more secure than embedding AWS_ACCESS_KEY_ID inside pods. Always use IRSA for pod-level AWS access in EKS.

Ingress on EKS

- Deploy AWS Load Balancer Controller (pod) in kube-system namespace
- Controller watches for Ingress resources via K8s API
- When Ingress is created, controller automatically creates ALB on AWS
- ALB sits in public subnets and routes to pods in private subnets

10. Load Balancers — ALB, NLB, GWLB

Type	OSI Layer	Sees	Best For
ALB	Layer 7	HTTP headers, URL paths, cookies	Microservices, REST APIs, web apps
NLB	Layer 4	TCP/UDP ports and IPs only	Gaming, VoIP, static IP, ultra-low latency
GWLB	Layer 3	Raw IP packets	Firewalls, IDS/IPS appliances
Interview Trap ALB terminates SSL — your backend only sees HTTP. NLB preserves source IP of clients. Use NLB when you need a static IP for firewall whitelisting.			

11. CI/CD — CodePipeline, CodeBuild, CodeDeploy

AWS Service	Role	Equivalent
CodeCommit	Managed Git repository	GitHub / GitLab
CodeBuild	Runs buildspec.yml — compile, test, build Docker image	Jenkins / GitHub Actions
CodeDeploy	Deploys to EC2/ECS/Lambda using appspec.yml	Ansible / Spinnaker
CodePipeline	Orchestrates full pipeline: source → build → deploy	GitLab CI / Jenkins Pipeline

buildspec.yml Structure (CodeBuild)

```

phases:  install:      runtime-versions:  python: 3.11  pre_build:      commands:
- docker login ... # login to registry  build:      commands:      - docker build -t
myapp .  post_build:      commands:      - docker push myapp

```

Store Docker credentials in SSM Parameter Store (SecureString) — reference in buildspec.yml under env.parameter-store.

12. CloudFormation Templates (CFT)

AWS-native IaC using YAML/JSON. Terraform = multi-cloud. CloudFormation = AWS-only but deeply integrated.

Term	Description
Template	YAML/JSON file describing infrastructure
Stack	A deployed instance of a template — all resources created together
Change Set	Preview changes before applying (like terraform plan)
Drift Detection	Detect when actual resource config differs from template
Parameters	Input values to make templates reusable

13. Route 53 — DNS Service

Routing Policy	When to Use
----------------	-------------

Simple	Single resource, no health checks
Weighted	A/B testing — split traffic 90%/10%
Latency-based	Route to region with lowest latency for user
Failover	Primary/secondary — auto fails to backup
Geolocation	Route based on user's country/continent

14. CloudFront — CDN

400+ edge locations worldwide. Without CDN: user in Pakistan hits US-East = high latency. With CloudFront: content cached at nearest edge (Mumbai).

- S3 + CloudFront: Keep S3 bucket PRIVATE. Only CloudFront accesses it via OAI (Origin Access Identity)
- DDoS protection via AWS Shield (integrated automatically)
- HTTPS at edge with ACM certificates

15. Secret Management

Service	Best For	Cost
SSM Parameter Store	Config values, Docker URLs, non-rotating secrets	Free (standard tier)
Secrets Manager	DB passwords, API keys that need auto-rotation	Paid per secret/month
HashiCorp Vault	Multi-cloud, enterprise, on-prem + cloud hybrid	Self-hosted cost

Cost Tip
Use Parameter Store where rotation is not needed — it's FREE. Secrets Manager charges per secret per month. Only use Secrets Manager when you need auto-rotation.

16. AWS Migration — The 7 Rs

Strategy	What Changes	When to Use
Rehost (Lift & Shift)	Move as-is to EC2	Fast migration, no code changes
Replatform	Use cloud-managed services (RDS, Multi-AZ)	Cloud benefits without code rewrite
Refactor	Redesign as cloud-native microservices	Need capabilities current arch can't deliver
Relocate	Move entire K8s cluster to EKS	Already on K8s, want managed control plane
Retain	Stay on-prem	Regulatory/compliance constraints
Retire	Decommission entirely	Unused or redundant applications
Repurchase	Replace with SaaS (Salesforce, etc.)	Mature SaaS does it better

Interview Trap

You do NOT need to break a monolith into microservices before migrating. Only Refactor requires this. Rehost, Replatform, and Repurchase all work fine with a monolith.

PART 3 — INTERVIEW Q&A

Terraform Interview Questions

Q1: What is Terraform and why use it over CloudFormation?

A: Terraform is a multi-cloud IaC tool. CloudFormation is AWS-only. Terraform works with AWS, Azure, GCP, and on-prem from the same code using HCL. It has a larger provider ecosystem and community.

Q2: What is a Terraform state file and why is it important?

A: terraform.tfstate is a JSON file tracking every resource Terraform manages. It tells Terraform what exists currently so it can calculate what to create/update/destroy on next apply. Without it, Terraform can't manage existing resources.

Q3: What is the chicken-and-egg problem with S3 backend?

A: You need S3 to store state, but Terraform uses state to create S3. Solution: First create S3 bucket without backend block, then add backend block and run terraform init again to migrate state to S3.

Q4: Difference between terraform plan and terraform apply?

A: Plan is a dry-run — shows what will change without making changes. Apply executes those changes. Always review plan before apply, especially in production.

Q5: What are Terraform modules and why use them?

A: Modules are reusable, encapsulated Terraform code in a folder with main.tf, variables.tf, outputs.tf. Benefits: reduce duplication, enforce consistency, easier team collaboration, versioning.

Q6: What are Terraform workspaces?

A: Workspaces let you manage multiple environments (dev/stage/prod) from one codebase, each with its own isolated state file. Switch with terraform workspace select prod.

Q7: What is remote backend and why needed?

A: Remote backend stores state file in S3/Terraform Cloud instead of locally. Benefits: team collaboration (shared state), state locking with DynamoDB to prevent concurrent applies, security (encrypted).

Q8: What is state locking and how is it implemented?

A: State locking prevents two people running terraform apply simultaneously, which could corrupt state. Implemented with a DynamoDB table — Terraform writes a lock record before apply and removes it after.

Q9: How do you handle sensitive values in Terraform?

A: Mark variables with sensitive = true (hides from console). Store in environment variables (TF_VAR_name). Use HashiCorp Vault data source to read secrets at runtime. Never hardcode secrets in .tf files.

Q10: What are provisioners? When should you avoid them?

A: file, remote-exec, local-exec — for bootstrapping instances. Terraform recommends avoiding them when possible (use User Data scripts or AMIs instead) because they run only on creation and failures are hard to handle.

Q11: What is the difference between count and for_each?

A: count creates N copies using an index (count = 3 creates 3 identical resources). for_each iterates over a map/set giving unique keys — better for creating similar-but-distinct resources with different names.

Q12: How does terraform import work?

A: Imports existing manually-created resources into Terraform state so Terraform can manage them going forward. terraform import aws_instance.example i-1234567890abcdef0

AWS Interview Questions

Q1: What is the difference between Security Group and NACL?

A: Security Group: instance level, stateful (response auto-allowed), allow rules only. NACL: subnet level, stateless (must allow both directions), allow AND deny rules, evaluated by rule number priority.

Q2: What is a NAT Gateway and when do you need it?

A: NAT Gateway lets private subnet instances make outbound internet calls (e.g., apt install, downloading packages) WITHOUT exposing them publicly. Traffic goes: private EC2 → NAT Gateway (in public subnet) → Internet.

Q3: Difference between IAM Role and IAM User?

A: User has permanent credentials (password or access keys). Role is temporary — it's ASSUMED by a service or user and issues short-lived STS tokens. Always use Roles for EC2, Lambda, and cross-account access.

Q4: What is IRSA in EKS?

A: IAM Roles for Service Accounts. Pods get temporary AWS credentials via OIDC token exchange without storing any access keys. Annotate K8s service account with IAM role ARN — pods using it auto-get credentials.

Q5: Difference between ALB and NLB?

A: ALB operates at Layer 7 (HTTP/HTTPS) — understands URL paths, headers, cookies. Good for microservices and web apps. NLB operates at Layer 4 (TCP/UDP) — ultra-fast, millions of req/sec, preserves source IP, supports static IP.

Q6: What are the 7 Rs of cloud migration?

A: Rehost (lift & shift), Replatform (minor cloud optimization), Refactor (redesign as cloud-native), Relocate (move K8s cluster), Retain (keep on-prem), Retire (decommission), Repurchase (switch to SaaS).

Q7: When to use Secrets Manager vs Parameter Store?

A: Parameter Store: config values, non-rotating secrets — FREE. Secrets Manager: DB passwords, API keys needing auto-rotation — costs money per secret/month. Use Parameter Store first unless rotation is required.

Q8: What is CloudFormation drift?

A: When actual resource configuration differs from what the CFT template says. Detect via Stack → Actions → Detect Drift. Common cause: someone manually changed a resource in the console outside of Terraform/CFT.

Q9: What is an Ingress Controller in EKS?

A: A pod (AWS Load Balancer Controller) that watches for Kubernetes Ingress resources. When you create an Ingress, the controller automatically creates and configures an ALB on AWS. It runs in kube-system namespace.

Q10: What is OIDC in EKS context?

A: OpenID Connect identity provider configured on the EKS cluster. It enables IRSA by allowing K8s service accounts to be trusted by IAM policies, enabling pods to assume IAM roles without storing credentials.

Q11: Fargate vs EC2 for ECS/EKS?

A: Fargate: no infrastructure management, auto-scales, pay per vCPU/memory used — great for simplicity. EC2: you manage instances, cheaper at scale, needed for GPU workloads or custom node configuration.

Q12: What is a VPC and why needed?

A: Virtual Private Cloud — your own isolated network inside AWS. Without a VPC your resources would be on shared public infrastructure. VPC gives you network isolation, custom IP ranges, subnet control, and security layers.

Q13: How does Auto Scaling work with ALB?

A: ALB distributes traffic across EC2 instances in a target group. Auto Scaling Group monitors CloudWatch metrics (CPU, request count). When threshold breached, ASG launches new instances and registers them with the ALB target group automatically.

Q14: What is an AMI and how is it used?

A: Amazon Machine Image — a snapshot/template of an OS + installed software. You choose an AMI when launching EC2. You can create custom AMIs from a configured instance for fast repeatable deployments (Golden Image pattern).

Q15: Explain the EBS Snapshot cleanup Lambda you built.

A: Lambda function that: gets all EBS snapshots owned by the account, gets all running EC2 instances, deletes snapshots with no associated volume, deletes snapshots whose volume isn't attached to any running instance. Scheduled via EventBridge cron.

PART 4 — PROJECT IDEAS (Portfolio & Upwork)

Tier 1 — Foundational Projects (Build These First)

These reinforce your existing learning and are quick to complete.

Project Name	What It Covers	Where to Show
Terraform + AWS VPC Full Network Setup	Creates VPC, public + private subnets across 2 AZs, IGW, NAT Gateway, route tables, security groups, bastion host. Parameterized with modules. Remote S3 backend with DynamoDB locking.	GitHub, CV, LinkedIn
Multi-Environment EC2 with Workspaces	One codebase, three Terraform workspaces (dev/stage/prod). Uses lookup() map for different instance types per env. Shows proper tfvars pattern.	GitHub, LinkedIn
Flask App Deployment with Terraform Provisioners	Terraform provisions EC2 inside VPC, copies app.py with file provisioner, installs Flask and starts app with remote-exec. App accessible via public IP.	GitHub, demo URL
S3 Remote Backend + DynamoDB State Locking	Full setup with chicken-and-egg solution documented. Team-ready Terraform state management. Good blog post material.	GitHub, Medium/Dev.to blog
EBS Snapshot Cleanup Lambda + EventBridge Cron	Lambda deletes stale snapshots nightly. Saves AWS costs. Includes CloudWatch logging, IAM role. Scheduled with EventBridge cron.	GitHub, Upwork

Tier 2 — Production-Grade Projects (Most Attractive)

These are the ones that impress hiring managers and Upwork clients.

Project Name	What It Covers	Where to Show
Complete AWS CI/CD Pipeline	GitHub → CodePipeline → CodeBuild (Docker image) → ECR → CodeDeploy to EC2. Secrets in SSM Parameter Store. Full IaC with Terraform.	LinkedIn, Upwork, CV
3-Tier AWS Architecture with Terraform	VPC + ALB (public) → EC2 Auto Scaling Group (private) → RDS Multi-AZ (private). Modules for each tier. CloudWatch alarms + SNS alerts. High availability design.	LinkedIn, Upwork, CV
EKS Cluster with ALB Ingress + IRSA	Terraform creates EKS cluster, Fargate profiles, AWS Load Balancer Controller via Helm, Ingress routing rules, OIDC + IRSA for pod AWS access. Deploy a real app.	LinkedIn, CV, GitHub
CloudFront + S3 Static Website with WAF	Terraform creates private S3 bucket, CloudFront distribution with OAI, ACM certificate for HTTPS, Route 53 record, WAF rules for DDoS protection.	Upwork, LinkedIn
Auto-Healing EC2 with CloudWatch + Lambda	CloudWatch alarm on CPU/health → triggers Lambda → Lambda terminates unhealthy EC2 → Auto Scaling Group replaces it. Full event-driven self-healing infra.	LinkedIn, blog, Upwork
Terraform Module Library	Reusable modules: vpc, ec2, rds, ecs-service, eks-cluster. Each with variables, outputs, README,	GitHub (pinned), Upwork, LinkedIn

	examples. Published to GitHub. This is extremely marketable.	
--	--	--

Tier 3 — Advanced / Upwork Goldmine Projects

These are niche, highly paid, and directly Upwork-marketable as services.

Project Name	What It Covers	Value
AWS Cost Optimization Automation	Lambda + EventBridge: delete unused EIPs, stop non-prod EC2 at night, right-sizing recommendations via Cost Explorer API, S3 lifecycle policies, unused snapshot cleanup. Full Terraform deployment.	Upwork \$\$\$ (very in-demand)
Kubernetes (EKS) Monitoring Stack	Terraform deploys EKS cluster + Prometheus + Grafana via Helm. Custom dashboards for pod CPU/memory, cluster health, alerts via Alertmanager → Slack/PagerDuty. Mirrors your on-prem Prometheus/Grafana skills on AWS.	LinkedIn, CV, Upwork
Multi-Account AWS Landing Zone	Terraform manages multiple AWS accounts with AWS Organizations. Centralized logging (CloudTrail → S3), centralized security (GuardDuty, SecurityHub), VPC peering between accounts, SCPs.	Enterprise clients, Upwork
GitOps Pipeline with ArgoCD on EKS	Terraform creates EKS + installs ArgoCD. Git repo = single source of truth for K8s manifests. Any push to Git auto-syncs to cluster. Add GitHub Actions for image build → ECR.	LinkedIn, very modern, attractive
Disaster Recovery Automation	Lambda + EventBridge: detect RDS/EC2 failures, trigger cross-region failover, update Route 53 DNS automatically. Test with chaos engineering (terminate instances). Terraform for all infra.	Upwork, enterprise clients
AWS Infrastructure Audit & Security Scanner	Python script + Lambda: checks for public S3 buckets, open security groups (0.0.0.0/0 on 22/3389), unencrypted EBS, root API key usage. Weekly report to SNS email. Terraform-deployed.	Upwork \$\$\$ (compliance, security)
Serverless API with DynamoDB + API Gateway	Terraform creates API Gateway → Lambda → DynamoDB. Lambda functions for CRUD. CloudWatch logging, X-Ray tracing. Custom domain with Route 53 + ACM. Complete with CI/CD.	Upwork, full-stack clients
Harbor → ECR Migration Tool	Script to pull images from your Harbor registry, retag, and push to ECR. Terraform creates ECR repos automatically. Documents migration path. Highly relevant to your current work.	LinkedIn, niche skill

Upwork Profile Strategy

Service Title	Why It Works
---------------	--------------

AWS DevOps Engineer — Terraform, EKS, CI/CD Pipelines	High search volume, specific skills
AWS Infrastructure Automation with Terraform	Cost optimization and cloud migration clients
Kubernetes (EKS/On-Prem) Setup and Monitoring	Your real competitive edge — on-prem + AWS hybrid
AWS Cost Optimization Automation	Every company wants this — very high ROI service
CI/CD Pipeline Setup — AWS CodePipeline / GitHub Actions	Constant demand from startups

Your Competitive Edge for Upwork/LinkedIn

Most AWS DevOps freelancers ONLY know cloud. You have 400+ node on-prem Kubernetes experience, Harbor registry, Prometheus/Grafana, GitLab CI — AND you are learning AWS. This hybrid background is RARE and valuable. Always frame it as: 'I manage production-grade on-premise Kubernetes clusters AND AWS infrastructure.' That combination wins enterprise clients.

QUICK REFERENCE — KEY FACTS TO MEMORIZE

Fact	Answer
S3 durability	99.999999999% (11 nines)
Lambda max timeout	15 minutes
EKS control plane cost	~\$0.10/hour per cluster
NACLs are ...	Stateless — must allow both inbound AND outbound
Security Groups are ...	Stateful — response traffic auto-allowed
IRSA stands for	IAM Roles for Service Accounts
OAI in CloudFront	Origin Access Identity — lets CloudFront read private S3
DynamoDB in Terraform backend	State locking — prevents concurrent terraform apply
Terraform workspace command	terraform workspace select <name>
S3 backend chicken-and-egg	Create bucket first without backend, then add backend and re-init
NAT Gateway purpose	Private subnet → outbound internet (NOT inbound)
ALB vs NLB layer	ALB = Layer 7 (HTTP). NLB = Layer 4 (TCP/UDP)
ECS Fargate vs EC2	Fargate = no infra management. EC2 = cheaper at scale
SSM vs Secrets Manager	SSM Parameter Store = free, no rotation. Secrets Manager = paid, auto-rotation
terraform.tfvars	Separate file for variable values — keep secrets here, not in .tf files